Instituto Tecnológico de Costa Rica

Escuela de Ingeniería Electrónica

Explicación de compilación de Toolchain `.bin`

Autor:

Iván Loaiza Arce

Fecha:

9 de marzo de 2026

Proyecto: Microcontrolador SIWA — TEC-RISC-V

Tema: Análisis de compatibilidad entre código compilado y arquitectura RV32I

Introducción

El presente documento describe el proceso de compilación, análisis y validación de un programa en lenguaje C para el microcontrolador **SIWA**, basado en la arquitectura **RISC-V RV32I**. El objetivo es demostrar que el código binario generado a partir del compilador `riscv64-linux-gnu-gcc` cumple con las características del conjunto de instrucciones y la organización de memoria del procesador SIWA.

Este proceso asegura que la interacción con periféricos mapeados en memoria, como el registro GPIO, se realiza correctamente utilizando únicamente instrucciones del conjunto base RV32I.

¿Qué es un Toolchain?

Un *toolchain* es un conjunto de herramientas de software que trabajan de forma encadenada para transformar un programa escrito en un lenguaje de alto nivel (como C o C++) en código máquina ejecutable por una arquitectura de procesador específica.

En el contexto de sistemas embebidos, el toolchain permite convertir el código fuente en instrucciones binarias que pueden cargarse directamente en la memoria del microcontrolador. Este proceso se realiza a través de varias etapas consecutivas, donde cada herramienta genera un artefacto intermedio que será utilizado por la siguiente herramienta de la cadena.

Para arquitecturas **RISC-V**, un toolchain típico está compuesto por las siguientes herramientas principales:

- **Compilador (`gcc`)** Convierte el código fuente escrito en C en código ensamblador específico de la arquitectura objetivo (en este caso, **RV32I**).
- **Ensamblador (`as`)** Traduce el código ensamblador en instrucciones máquina y genera archivos objeto intermedios.
- **Enlazador (`ld`)** Combina los archivos objeto generados y organiza las secciones del programa (`.text`, `.data`, `.bss`) en el mapa de memoria definido para el sistema.
- **Herramientas de conversión y análisis** Utilidades como `objcopy` y `objdump` permiten convertir los archivos ejecutables a diferentes formatos binarios (`.hex`, `.bin`) y analizar el contenido de las instrucciones generadas.

En este proyecto se utiliza un toolchain basado en **GNU GCC para RISC-V**, el cual permite generar programas compatibles con el conjunto de instrucciones **RV32I** implementado en el microcontrolador **SIWA**.

La Figura conceptual del flujo de compilación puede resumirse de la siguiente manera:

`Código C` → `Ensamblador (.s)` → `Ejecutable (.elf)` → `Binario (.bin / .hex)`

Cada uno de estos artefactos intermedios permite analizar diferentes niveles de abstracción del programa, desde las instrucciones ensamblador generadas por el compilador hasta el formato binario final que será cargado en la memoria del sistema.

Primera etapa del toolchain: compilación del código fuente

La primera etapa del toolchain corresponde al proceso de compilación, en el cual el código fuente escrito en lenguaje C es analizado y traducido a código ensamblador específico para la arquitectura objetivo.

Durante esta etapa, el compilador interpreta las estructuras del programa, como variables, operaciones aritméticas y accesos a memoria, y las transforma en instrucciones de bajo nivel compatibles con el procesador. En el proyecto SIWA, el objetivo es generar instrucciones del conjunto base de la arquitectura RISC-V RV32I, que corresponde al ISA implementado por el microcontrolador.

El resultado de esta etapa es un archivo con extensión `.s`, que contiene el programa en lenguaje ensamblador. Este archivo permite analizar cómo el compilador traduce el código en C a instrucciones del procesador antes de continuar con las etapas de ensamblado y enlazado del toolchain.

Validación del código según las restricciones de SIWA

Antes de iniciar el proceso de compilación, la herramienta SIWA TOOLCAHIN GUI realiza un análisis estático del código fuente con el objetivo de verificar que el programa sea compatible con las limitaciones de la arquitectura del microcontrolador SIWA. Este análisis se realiza leyendo el archivo fuente línea por línea y aplicando diferentes reglas que detectan construcciones no permitidas dentro del entorno del sistema.

El sistema SIWA opera en modo *bare-metal*, es decir, sin sistema operativo ni soporte completo de bibliotecas estándar del lenguaje C. Por esta razón, ciertas funciones y comportamientos del código deben ser restringidos antes de iniciar la compilación.

Una de las primeras restricciones implementadas corresponde a la prohibición de funciones de la biblioteca estándar que no están disponibles en el sistema. Estas funciones se definen en una lista llamada `PROHIBIDAS` dentro del programa.

```
PROHIBIDAS = ["printf", "scanf", "malloc", "free", "fopen", "fgets", "fprintf"]
```

Durante el análisis del código fuente, cada línea es revisada para determinar si contiene alguna de estas funciones. En caso afirmativo, la herramienta genera un mensaje de error indicando la línea donde se encontró la función prohibida.

```
for funcion in PROHIBIDAS:
    if re.search(rf"\b{funcion}\b", linea):
        errores.append(f"[Línea {num}] Uso de función prohibida: {funcion}")
```

Otra verificación importante corresponde al uso de punteros hacia direcciones de memoria. Cuando se accede a registros de hardware mapeados en memoria, estos deben declararse utilizando el modificador `volatile`, para evitar que el compilador optimice el acceso al periférico.

El siguiente fragmento del código detecta punteros declarados sin el modificador `volatile`.

```
if re.search(r"\bint\s*\*\s*\w+", linea) and "volatile" not in linea:
    errores.append(f"[Línea {num}] Puntero sin 'volatile': {linea.strip()}")
```

La herramienta también valida las direcciones de memoria utilizadas en el programa. Para ello se define una función llamada `es_direccion_valida()`, la cual verifica si una dirección hexadecimal pertenece a alguno de los rangos permitidos por la arquitectura SIWA.

```
def es_direccion_valida(valor: str) -> bool:
    try:
        direccion = int(valor, 16)
        return (0x00000000 <= direccion <= 0x00001FFF) \
            or (direccion >= 0x00800000) \
            or (direccion >= 0x80000000)
    except ValueError:
        return False
```

Durante el análisis del código, todas las direcciones hexadecimales encontradas son evaluadas mediante esta función. Si alguna dirección no pertenece a los rangos permitidos, la herramienta genera un error.

```
matches = re.findall(r"0x[0-9a-fA-F]+", linea)
for match in matches:
    if not es_direccion_valida(match):
        errores.append(f"[Línea {num}] Dirección no válida: {match}")
```

Adicionalmente, el programa verifica que no se utilicen instrucciones que no estén soportadas por la arquitectura RV32I implementada en el procesador SIWA. Entre estas instrucciones se encuentran `fence` y `fence.i`, las cuales son detectadas directamente durante el análisis del código.

```
if "fence.i" in linea or re.search(r"\bfence\b", linea):
    errores.append(f"[Línea {num}] Instrucción no soportada: {linea.strip()}")
```

Finalmente, el sistema revisa el uso de registros de control y estado (CSR). En la arquitectura SIWA se permite únicamente un rango limitado de registros CSR, por lo que cualquier acceso fuera de este rango genera un error durante la validación del código.

```
if re.search(r"\bcsrr\w*\b", linea):
    m = re.search(r"0x[0-9a-fA-F]+", linea)
    if m and int(m.group(), 16) > 0x07:
        errores.append(f"[Línea {num}] CSR fuera del rango permitido: {m.group()}")
```

Esta etapa de validación permite detectar incompatibilidades antes de iniciar el proceso de compilación. De esta manera se evita generar archivos binarios inválidos y se garantiza que el programa cumpla con las restricciones impuestas por la arquitectura del microcontrolador SIWA.

Flujo de análisis y compilación en la herramienta SIWA TOOLCAHIN GUI

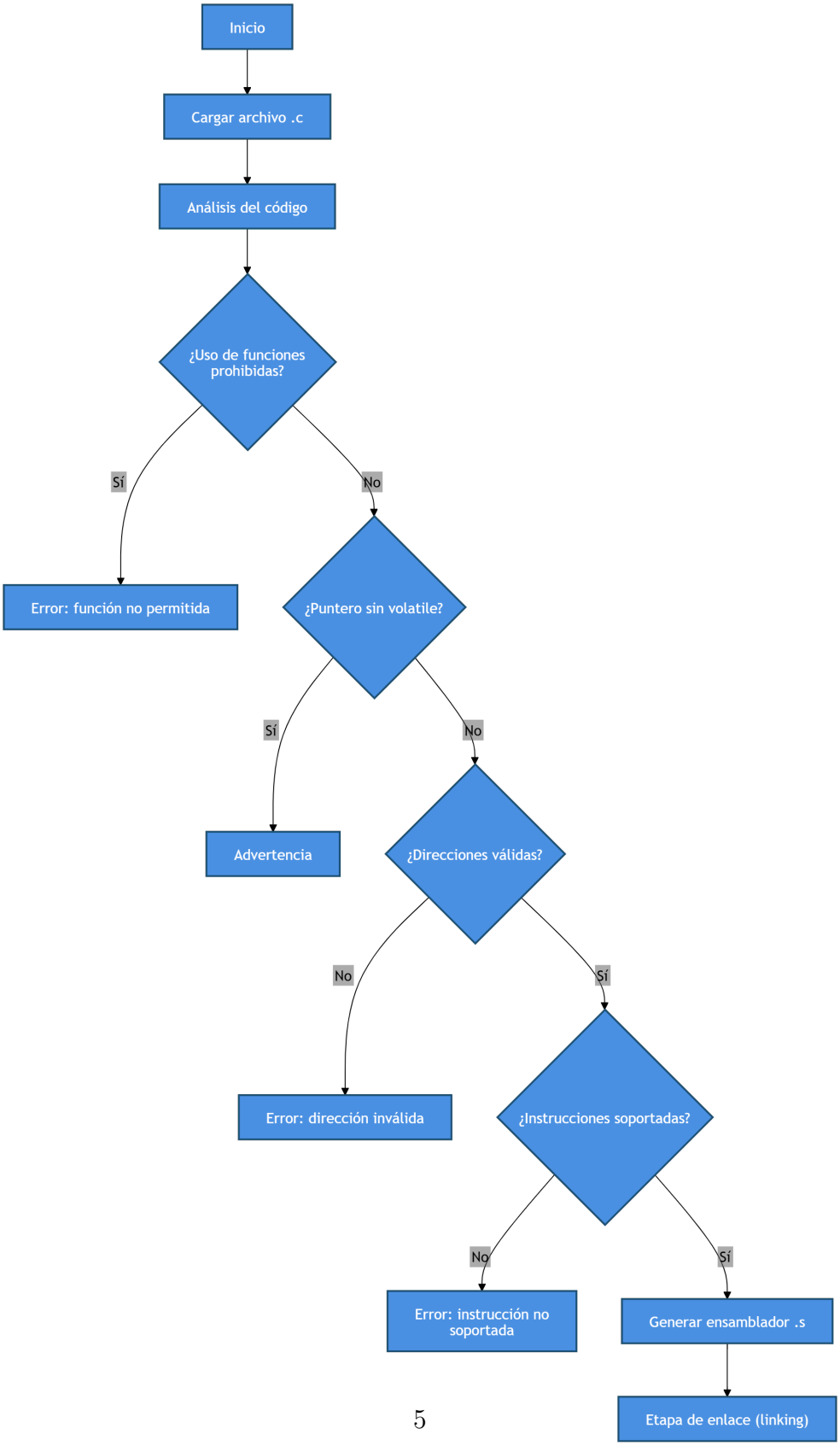

Figura 1: Diagrama de flujo del proceso de validación y compilación en SIWA.

Compilación del código fuente para generar el archivo `.s`

Luego de la etapa de validación, la herramienta continúa con la primera fase formal del toolchain, correspondiente a la compilación del código fuente en lenguaje C para generar un archivo en ensamblador con extensión `.s`. Esta etapa permite traducir el programa de alto nivel a un conjunto de instrucciones representadas en lenguaje ensamblador, de acuerdo con la arquitectura objetivo del sistema.

En el proyecto SIWA, esta compilación se realiza para la arquitectura RISC-V RV32I, que corresponde al conjunto base de instrucciones soportado por el microcontrolador. Para garantizar esta compatibilidad, la herramienta configura el compilador cruzado con los parámetros `-march=rv32i` y `-mabi=ilp32`. Además, utiliza la opción `-S`, la cual indica que el proceso debe detenerse después de generar el archivo ensamblador, sin pasar todavía a la etapa de enlace.

Dentro del programa, esta operación se implementa en el método `on_build()`, donde primero se define el nombre del compilador, se obtienen los parámetros seleccionados por el usuario y luego se construye el comando que será ejecutado desde la interfaz.

El fragmento de código responsable de generar el archivo `.s` es el siguiente:

```
# 1) .s
if self.want_s.get():
    cmd = [gcc, f"-march={march}", f"-mabi={mabi}", "-S", src, "-o", f"{base}.s"]
    self._run_and_log("Generando ensamblador (.s)", cmd)
```

En este fragmento, la variable `self.want_s` permite verificar si el usuario ha seleccionado la opción de generar el archivo ensamblador. Si esta opción está activa, la herramienta construye el comando de compilación en la lista `cmd`.

Cada parte del comando cumple una función específica:

- `gcc`: corresponde al compilador cruzado definido a partir del prefijo del toolchain, por ejemplo `riscv64-unknown-elf-gcc`.
- `-march=rv32i`: obliga al compilador a generar únicamente instrucciones del conjunto base RV32I.
- `-mabi=ilp32`: establece el modelo de datos de 32 bits utilizado por la arquitectura.
- `-S`: indica que el resultado de esta etapa debe ser un archivo ensamblador.
- `src`: representa el archivo fuente en C seleccionado por el usuario.
- `-o f"{base}.s"`: define el nombre del archivo de salida con extensión `.s`.

De forma general, el comando ejecutado por la herramienta tiene la siguiente estructura:

```
riscv64-unknown-elf-gcc -march=rv32i -mabi=ilp32 -S programa.c -o programa.s
```

Una vez construido el comando, la herramienta lo ejecuta mediante la función `_run_and_log()`, la cual se encarga de invocar el compilador y mostrar en la consola de la interfaz tanto el comando ejecutado como su resultado.

```
def _run_and_log(self, title: str, cmd: list[str]):
    self._log(f" {title}\n$ {' '.join(cmd)}\n")
    rc, out = run_cmd(cmd)
    self._log(out + ("\n" if not out.endswith("\n") else ""))
    if rc == 0:
        self._log(" OK\n\n")
    else:
        self._log(f" Error (returncode={rc})\n\n")
```

La ejecución real del comando se realiza por medio de la función `run_cmd()`, la cual utiliza el módulo `subprocess` de Python para llamar al compilador instalado en el sistema.

```
def run_cmd(cmd: list[str]) -> tuple[int, str]:
    try:
        p = subprocess.run(cmd, stdout=subprocess.PIPE,
                           stderr=subprocess.STDOUT,
                           text=True, check=False)
        return p.returncode, p.stdout
    except FileNotFoundError:
        return 127, f"Comando no encontrado: {cmd[0]}"
```

Como resultado de esta etapa, se genera un archivo con extensión `.s`, el cual contiene el programa traducido a ensamblador RISC-V. Este archivo permite observar cómo el compilador transforma las operaciones del código en C, como sumas, accesos a memoria y estructuras de control, en instrucciones de bajo nivel compatibles con el procesador.

Por tanto, la generación del archivo `.s` constituye la primera salida formal del proceso de compilación y representa el punto de transición entre el programa escrito por el usuario y su representación en lenguaje ensamblador, previo a la etapa de enlace del ejecutable.

Etapa de enlace para generar el archivo `.elf`

Después de la generación del archivo ensamblador `.s`, el siguiente paso del toolchain corresponde a la etapa de enlace. En esta fase, el objetivo es construir el archivo ejecutable final con extensión `.elf`, organizando correctamente las diferentes secciones del programa dentro del mapa de memoria definido para la arquitectura SIWA.

De forma conceptual, el enlazador toma el código ensamblador generado en la etapa anterior y ubica sus secciones en direcciones concretas de memoria. En particular, coloca el código ejecutable en la región destinada a instrucciones, los datos inicializados en la región de datos, y las variables no inicializadas en la sección correspondiente a `.bss`. De esta manera, el archivo `.elf` ya no es solamente una traducción simbólica del programa, sino una imagen ejecutable con una organización de memoria definida.

En la herramienta SIWA TOOLCAHIN GUI , esta etapa se realiza dentro del método `on_build()`, donde se construye el comando del compilador para generar el archivo `.elf`. Si el usuario proporciona un archivo `linker.ld`, la herramienta utiliza dicho archivo para guiar el proceso de enlace. El fragmento del código responsable de esta operación es el siguiente:

```
if self.want_elf.get():
    if linker:
        cmd = [
            gcc, f"-march={march}", f"-mabi={mabi}",
            "-nostdlib", "-ffreestanding", "-fno-builtin",
            "-T", linker,
            "-o", f"{base}.elf",
            f"{base}.s" if os.path.exists(f"{base}.s") else src
        ]
```

En este bloque, la variable `self.want_elf` verifica si el usuario seleccionó la opción de generar el archivo ejecutable. Si además existe un archivo `linker.ld`, entonces se usa la opción `-T` para indicar al compilador que debe enlazar el programa según el mapa de memoria definido en ese script.

Los parámetros utilizados en esta etapa tienen funciones específicas:

- `-nostdlib`: evita enlazar bibliotecas estándar del sistema.
- `-ffreestanding`: indica que el entorno es bare-metal y no un sistema hospedado.
- `-fno-builtin`: impide que el compilador reemplace funciones por versiones internas.
- `-T linker.ld`: le indica al enlazador que utilice el script de enlace especificado.

- `-o archivo.elf`: define el nombre del ejecutable de salida.

Con esta configuración, la herramienta genera un archivo `.elf` adaptado a la arquitectura SIWA, respetando la organización de memoria definida por el proyecto.

Uso del linker script en la herramienta

Si no se proporciona un archivo `linker.ld`, la herramienta usa un método alternativo para ubicar el código mediante la opción `-Wl,-Ttext`. Esto se observa en el siguiente fragmento:

```
else:
    cmd = [
        gcc, f"-march={march}", f"-mabi={mabi}",
        "-nostdlib", "-ffreestanding", "-fno-builtin",
        f"-Wl,-Ttext={addr_text}",
        "-o", f"{base}.elf",
        f"{base}.s" if os.path.exists(f"{base}.s") else src
    ]
```

En este caso, únicamente se fija la dirección de inicio de la sección `.text`, pero no se define de manera completa la distribución de las demás secciones del programa. Por esta razón, el uso de un `linker.ld` resulta más apropiado, ya que permite un control mucho más preciso sobre la ubicación del código, los datos y las regiones de periféricos.

Función del linker script

El archivo `linker.ld` define cómo deben ubicarse las secciones del programa en la memoria del sistema. En otras palabras, establece el mapa de memoria que el enlazador utilizará para construir el ejecutable final.

El script utilizado en el proyecto es el siguiente:

```
ENTRY(main)  /* Define el punto de entrada del programa en la función main */

MEMORY {
  CODE (rx)  : ORIGIN = 0x00000000, LENGTH = 4K    #Región para código ejecutable:
                                                     lectura y ejecución, inicia en
                                                     0x00000000 y tiene 4 KB

  DATA (rw)  : ORIGIN = 0x00001000, LENGTH = 4K   #Región para datos:
                                                     lectura y escritura, inicia en
```

```
                                                    0x00001000 y tiene 4 KB

  UART (rw)  : ORIGIN = 0x0080000, LENGTH = 8M    #Región reservada para UART:
                                                    lectura y escritura, inicia en
                                                    0x0080000 y tiene 8 MB

  SPI  (rw)  : ORIGIN = 0x0100000, LENGTH = 16M   #Región reservada para SPI:
                                                    lectura y escritura, inicia en
                                                    0x0100000 y tiene 16 MB
}

SECTIONS {

  .text : {                                       #Sección de código ejecutable
                                                    y datos de solo lectura
    *(.text*)                                     #Incluye todas las subsecciones
                                                    de código
    *(.init)                                      #Incluye rutinas de
    *(.rodata*)                                   #Incluye constantes y datos
                                                    de solo lectura
    KEEP(*(.startup*))                            #Conserva las secciones de arranque
                                                    aunque haya optimización
  } > CODE                                        #Ubica esta sección en la región CODE

  .data : {                                       #Sección para datos inicializados
    *(.data*)                                     #Incluye todas las subsecciones
                                                    de datos inicializados
  } > DATA                                        #Ubica esta sección en la región DATA

  .bss : {                                        #Sección para datos no inicializados
    *(.bss*)                                      #Incluye variables globales
                                                    sin inicializar
    *(COMMON)                                     #Incluye variables comunes
  } > DATA                                        #Ubica esta sección también en DATA

  .uart : {                                       #Sección específica para UART
    *(.uart*)                                     #Incluye cualquier dato o sección
```

```
                                                    etiquetada para UART
  } > UART                                        #Ubica esta sección en la región UART

  .spi : {                                        #Sección específica para SPI
    *(.spi*)                                      #Incluye cualquier dato o sección
                                                    etiquetada para SPI
  } > SPI                                         #Ubica esta sección en la región SPI
}
```

La directiva `ENTRY(main)` define el símbolo de entrada del programa. Esto significa que la ejecución comenzará en la función `main`. En la práctica, esta configuración puede funcionar para programas simples, aunque en sistemas embebidos frecuentemente se utiliza un símbolo de arranque como `_start` junto con un archivo de inicio. En este script, el punto de entrada fue definido directamente en `main`.

La sección `MEMORY` define las regiones de memoria disponibles para el programa. En este caso, se separan las áreas de código, datos y dos regiones adicionales para periféricos.

- `CODE`: región de solo lectura y ejecución ubicada en la dirección `0x00000000`, destinada al código del programa.
- `DATA`: región de lectura y escritura ubicada en `0x00001000`, destinada a variables globales y datos.
- `UART`: región reservada para secciones asociadas al periférico UART.
- `SPI`: región reservada para secciones asociadas al periférico SPI.

La sección `SECTIONS` indica cómo se deben mapear las partes del programa dentro de esas regiones.

La sección `.text` agrupa el código ejecutable, las rutinas de inicialización y los datos de solo lectura. La instrucción `*(.text*)` recoge todas las subsecciones de código; `*(.init)` incluye rutinas de inicialización; y `*(.rodata*)` almacena constantes de solo lectura. Además, `KEEP(*(.startup*))` asegura que cualquier sección de arranque no sea eliminada por optimización. Toda esta información se ubica en la región `CODE`.

La sección `.data` contiene los datos inicializados del programa y se coloca en la región `DATA`. Por su parte, la sección `.bss` agrupa las variables no inicializadas y también se ubica en la región `DATA`. Esto permite que tanto variables globales con valor inicial como variables sin inicializar queden dentro del espacio de datos del sistema.

Finalmente, las secciones `.uart` y `.spi` permiten ubicar información específica en regiones asociadas a periféricos concretos. Aunque no siempre se usan en programas sencillos, su

presencia en el script facilita reservar zonas del mapa de memoria para componentes del sistema embebido.

Resultado de la etapa de enlace

Como resultado de esta fase, se obtiene un archivo `.elf` que contiene el programa ya enlazado y organizado según el mapa de memoria de SIWA. Este archivo sirve como base para las etapas posteriores del toolchain, como la conversión a `.hex` y `.bin`, y además puede emplearse para simulación en entornos como QEMU eso sin utilizar el linker ya que este define la estructura de memoria del SIWA y QUEMU utiliza la del computador.

Por tanto, la etapa de enlace no solo une el código generado en la fase de compilación, sino que también establece la correspondencia definitiva entre las secciones del programa y la memoria física o lógica del sistema, lo cual es fundamental para garantizar la correcta ejecución del software sobre la arquitectura SIWA.

Herramientas de conversión y análisis

Una vez generado el archivo ejecutable `.elf`, el siguiente paso del toolchain consiste en utilizar herramientas de conversión y análisis que permiten obtener otros formatos útiles para carga en memoria, simulación y verificación del programa. En esta etapa intervienen principalmente utilidades como `objcopy` y, de manera conceptual, también herramientas como `objdump` para inspección del contenido binario.

La función de estas herramientas es transformar el archivo ejecutable `.elf`, que contiene información estructurada sobre el programa, en formatos más simples y directos como `.hex` y `.bin`. Estos formatos son especialmente útiles en sistemas embebidos, ya que permiten inicializar memorias, cargar datos en FPGA o trasladar el programa a entornos de simulación y prueba.

En la herramienta SIWA TOOLCAHIN GUI , esta etapa se implementa después de la generación del archivo `.elf`. Para ello, el programa utiliza la herramienta `objcopy` del toolchain RISC-V, encargada de convertir el ejecutable a otros formatos.

Generación del archivo `.hex`

El archivo `.hex` se produce en formato HEX, el cual representa el contenido del programa en texto hexadecimal estructurado. Este formato resulta útil para inicialización de memorias y para visualizar el contenido binario de forma ordenada.

En el código Python, la generación del archivo `.hex` se realiza de la siguiente manera:

```
if self.want_hex.get():
    if not os.path.exists(f"{base}.elf"):
        self._log(" No existe ELF. Genera primero .elf.\n")
    else:
        cmd = [objcopy, "-O", "ihex", f"{base}.elf", f"{base}.hex"]
        self._run_and_log("Generando HEX (.hex)", cmd)
```

En este fragmento, la variable `self.want_hex` indica si el usuario seleccionó la opción de generar el archivo `.hex`. Antes de ejecutar la conversión, la herramienta verifica que el archivo `.elf` exista, ya que este es la entrada necesaria para el proceso. Si el ejecutable está disponible, se construye el comando de conversión utilizando `objcopy` con la opción `-O ihex`, la cual especifica que la salida debe producirse en formato HEX.

De manera general, el comando ejecutado tiene la forma:

```
riscv64-unknown-elf-objcopy -O ihex programa.elf programa.hex
```

Generación del archivo `.bin`

El archivo `.bin` corresponde a un binario plano, es decir, una secuencia directa de bytes sin metadatos adicionales. Este formato es especialmente importante en sistemas embebidos, ya que permite cargar el contenido del programa directamente en memoria o utilizarlo como imagen binaria en pruebas de hardware.

En la herramienta, la conversión a `.bin` se implementa con el siguiente código:

```
if self.want_bin.get():
    if not os.path.exists(f"{base}.elf"):
        self._log(" No existe ELF. Genera primero .elf.\n")
    else:
        cmd = [objcopy, "-O", "binary", f"{base}.elf", f"{base}.bin"]
        self._run_and_log("Generando BIN (.bin)", cmd)
```

Aquí, la variable `self.want_bin` permite verificar si el usuario ha seleccionado la generación del archivo binario. Si el archivo `.elf` existe, la herramienta construye el comando con la opción `-O binary`, la cual le indica a `objcopy` que debe generar un binario plano a partir del ejecutable.

El comando general correspondiente es:

```
riscv64-unknown-elf-objcopy -O binary programa.elf programa.bin
```

Ejecución de los comandos de conversión

Tanto en la generación de `.hex` como de `.bin`, la ejecución real del comando se realiza mediante la función `_run_and_log()`, la cual muestra en la consola de la interfaz el comando ejecutado y su resultado.

```
def _run_and_log(self, title: str, cmd: list[str]):
    self._log(f" {title}\n$ {' '.join(cmd)}\n")
    rc, out = run_cmd(cmd)
    self._log(out + ("\n" if not out.endswith("\n") else ""))
    if rc == 0:
        self._log(" OK\n\n")
    else:
        self._log(f" Error (returncode={rc})\n\n")
```

Esta función llama internamente a `run_cmd()`, que utiliza el módulo `subprocess` para invocar la herramienta correspondiente del toolchain.

```
def run_cmd(cmd: list[str]) -> tuple[int, str]:
    try:
        p = subprocess.run(cmd, stdout=subprocess.PIPE,
                           stderr=subprocess.STDOUT,
                           text=True, check=False)
        return p.returncode, p.stdout
    except FileNotFoundError:
        return 127, f" Comando no encontrado {cmd[0]}\nAsegúrate de que esté en el
        PATH."
```

Herramientas de análisis

Además de la conversión de formatos, en esta etapa también se pueden emplear herramientas de análisis para inspeccionar el contenido del programa generado. Una de las más importantes es `objdump`, la cual permite desensamblar el binario o el ejecutable y observar las instrucciones máquina generadas por el compilador y el enlazador.

Aunque en la versión actual de la herramienta Python no se incluyó una llamada directa a `objdump`, dentro del flujo general del proyecto sí se utiliza conceptualmente como apoyo para verificar que el binario contenga únicamente instrucciones compatibles con la arquitectura RISC-V RV32I. Por ejemplo, un comando de análisis como el siguiente permite inspeccionar el binario generado:

```
riscv64-unknown-elf-objdump -D programa.elf
```

o bien, en el caso de un binario plano:

```
riscv64-linux-gnu-objdump -D -b binary -m riscv:rv32 programa.bin
```

Estas herramientas de análisis son útiles para confirmar que el código final preserve la compatibilidad con la arquitectura SIWA, verificar accesos a memoria, observar instrucciones de carga y almacenamiento, e inspeccionar el control de flujo del programa.

Resultado de la etapa

Como resultado de esta fase, el sistema genera representaciones alternativas del programa ya enlazado. El archivo `.hex` ofrece una representación hexadecimal estructurada, mientras que el archivo `.bin` proporciona una imagen plana de bytes lista para carga directa en memoria. Junto con las herramientas de análisis, esta etapa permite no solo preparar el software para su implementación en hardware, sino también verificar técnicamente el contenido del ejecutable generado por el toolchain.

Conclusión

El análisis realizado permite concluir que el funcionamiento del toolchain en el proyecto SIWA se desarrolla como una secuencia estructurada de etapas que transforman un programa escrito en lenguaje C en archivos ejecutables y binarios compatibles con la arquitectura RISC-V RV32I.

En primer lugar, la herramienta SIWA TOOLCAHIN GUIejecuta una etapa de validación previa del código fuente, en la cual se verifica que el programa cumpla con las restricciones propias del microcontrolador SIWA. Esta revisión permite detectar, antes de compilar, el uso de funciones no soportadas, direcciones inválidas, accesos inadecuados a periféricos e instrucciones incompatibles con la arquitectura implementada. De esta forma, se evita que el proceso continúe con código que podría generar errores de ejecución o binarios no válidos para el sistema.

Una vez superada esta validación, se realiza la primera etapa formal del toolchain, correspondiente a la compilación del código fuente para generar el archivo ensamblador `.s`. En esta fase, el compilador traduce las estructuras del lenguaje C a instrucciones de bajo nivel compatibles con la arquitectura RV32I, permitiendo observar con mayor detalle cómo el programa será interpretado por el procesador.

Posteriormente, el proceso continúa con la etapa de enlace, en la cual se genera el archivo ejecutable `.elf`. En esta fase, el enlazador organiza las diferentes secciones del programa

dentro del mapa de memoria definido para SIWA, ubicando el código, los datos inicializados y las variables no inicializadas en sus regiones correspondientes. El uso del archivo `linker.ld` resulta fundamental en esta etapa, ya que permite establecer de forma explícita la estructura de memoria del sistema y asegurar que el ejecutable final responda a la organización interna de la arquitectura.

Finalmente, a partir del archivo `.elf`, las herramientas de conversión generan formatos adicionales como `.hex` y `.bin`, los cuales facilitan la carga del programa en memoria, su uso en FPGA y la preparación de imágenes binarias para pruebas o implementación. De manera complementaria, herramientas de análisis como `objdump` permiten inspeccionar el contenido del ejecutable y verificar que las instrucciones generadas sean coherentes con el conjunto base RV32I.

En conjunto, el toolchain implementado en el proyecto no solo permite convertir código fuente en un programa ejecutable para el microcontrolador SIWA, sino que también incorpora mecanismos de validación, organización y análisis que fortalecen la confiabilidad del proceso. Esto demuestra que la integración entre compilador, enlazador y herramientas de conversión constituye un flujo completo y coherente para el desarrollo de software sobre la arquitectura TEC-RISC-V, asegurando compatibilidad entre el código generado y las características funcionales del sistema.

Autor: Iván Loaiza Arce
Institución: Instituto Tecnológico de Costa Rica